



Assignment 1

Name: Matanat Heydarli 18080

Instructor: Nurlan Imanov

CRN: 20913

Submission date: 05.04.2026

1. Part 1

Introduction:

For this part of the assignment, I had to install Apache Hadoop (Hadoop 3.x) on my system and also verify whether Hadoop is working correctly in standalone mode. Hadoop is an open-source system that is used for storing and processing huge amounts of data. Unlike other systems that need extremely powerful computers, Hadoop is designed in a manner that is distributed. This makes Hadoop scalable and cost-effective. Although in this task, I have used standalone mode on a single machine, installing Hadoop on my system helped me understand the basic architecture of Hadoop and how it works. I have completed all these tasks on my MacBook Air M4.

Installing Java 8:

Firstly, I should install Java 8 on my system. I tried to use Homebrew for *openjdk@8* installation on my laptop, but the installation failed because Homebrew required x86_64 architecture which my Apple Silicon does not support. This was the first technical issue I faced.

As a solution, I had to download and install the ARM-compatible Java 8 version which I chose Azul Zulu Java 8. After completing the installation process, I checked if Java existed on the system. I set up the Java path in my shell configuration so the terminal would access the appropriate Java version.

I added the following to my .zshrc file:

```
export JAVA_HOME=/Library/Java/JavaVirtualMachines/zulu-8.jdk/Contents/Home
export PATH="$JAVA_HOME/bin: $PATH"
```

In this command, the JAVA_HOME variable tells the system where Java is installed. The system requires PATH changes which enable direct execution of Java commands from the terminal. I executed `java -version` command to verify that the system had installed Java 8 correctly. This step is important because Hadoop requires Java to function correctly. Hadoop commands will

fail if the system lacks Java or runs an incorrect Java version even though Hadoop itself functions properly.

Downloading Hadoop:

I downloaded Hadoop 3.3.6 from the official Apache Hadoop website after Java was ready I selected version 3.3.6 because it satisfies the assignment requirement of Hadoop 3.x and works well with Java 8. When I downloaded the archive on macOS, I noticed that Safari handled the filename differently than expected. The downloaded file appeared as “.tar” because it did not allow normal “.tar.gz” file operations. First time, this situation occurred when my extraction command failed because I attempted to use an incorrect filename like “hadoop-3.3.6.tar.gz”. I extracted the downloaded file after checking it thoroughly with the following command:

```
tar -xf hadoop-3.3.6.tar
```

I moved the extracted Hadoop folder into my home directory.

Setting Hadoop Environment:

The next step was to make Hadoop commands available from the terminal. So, I edited the .zshrc file again and added Hadoop environment variables:

```
export HADOOP_HOME=~/.hadoop-3.3.6
export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HOME/sbin
```

Then I reloaded the shell configuration using:

```
source ~/.zshrc
```

This step allowed me to run commands such as Hadoop version directly from the terminal.

Configuring Hadoop:

After that, I configured Hadoop itself. The hadoop-env.sh file required me to configure Hadoop to recognize Java installation paths. I searched for a pre-existing export JAVA_HOME= statement in the file because I expected to find it there. The Hadoop version I used at that time did not include the expected line, but I added it manually to the file. I added the correct Java path to the file by entering this line to the end:

```
export JAVA_HOME=/Library/Java/JavaVirtualMachines/zulu-8.jdk/Contents/Home
```

I made a mistake when I copied terminal output to the configuration file while I was working on that file. The syntax error appeared when I attempted to execute Hadoop. I had to inspect the file, identify where the pasted terminal text started, delete all the invalid lines, and keep only proper configuration content. While debugging, I realised that a single incorrect line which I pasted into a configuration file could disable program operation. I used hadoop version command to test Hadoop installation after completing installation and configuration. The command displayed the Hadoop version information successfully. This confirmed that Hadoop 3.3.6 was installed and that the Java configuration was working correctly. The setup operated

in standalone mode which enables Hadoop to execute on a single machine instead of a multi-node cluster.

Verifying with Examples:

To make sure that Hadoop was not only installed but also working properly, I did tests. I did 2 tests. The first test I did was the built-in example of a grep program. I created an input directory and then copied Hadoop's XML configuration files into this directory. I then ran a grep program that would count the word "configuration".

The command was successful, and the result was:

27 configurations

This proved that Hadoop was working as expected, reading the input files and generating the desired output.

Then, I did second test. This time I used the wordcount example. I created a file that looked like this:

Introduction to big data

big data assignment

quiz from big data

I then carried out the wordcount example and got the results:

big 3

data 3

introduction 1

to 1

assignment 1

quiz 1

from 1

This proved that Hadoop was working as expected, and I got exactly the results I expected.

Challenges and Solutions:

The first problem I encountered was when I was installing Java 8. The problem was that the package I was installing required the x86_64 architecture, but my MacBook Air M4 uses the Apple Silicon chip, which is ARM-based. This problem happened at first because most tutorials that provide instructions on how to install Java and other software do not mention this. I solved problem when I found the ARM-based Java 8 package and installed Azul Zulu Java 8.

The second problem I encountered was when I was installing Hadoop. The problem was that the Hadoop package I downloaded was in the .tar file format instead of the .ar.gz I was expecting.

However, after checking the package name, I realized that the problem was caused by the way the.tar.gz file was extracted to .tar.

One of my biggest challenges was with hadoop-env.sh. I mistakenly pasted some lines of code from my terminal. When I tried to run Hadoop version, Hadoop reported that there was a syntax error. However, I noticed that the lines reported were not even in my configuration file. This challenge was not Hadoop's fault but mine. To fix it, I opened my hadoop-env.sh file, searched for the lines that I had pasted by mistake, deleted them, and left only valid lines. After doing that, my Hadoop version command worked correctly.

Another challenge that I had was that Hadoop had to be configured with Java's path. Although Java 8 was installed in my system and could be run using my terminal, Hadoop still had to be configured with Java's path. However, I thought that Hadoop's configuration would have JAVA_HOME. But it did not. Therefore, I had to manually set JAVA_HOME in my hadoop-env.sh. After doing that, Hadoop's commands were working correctly.

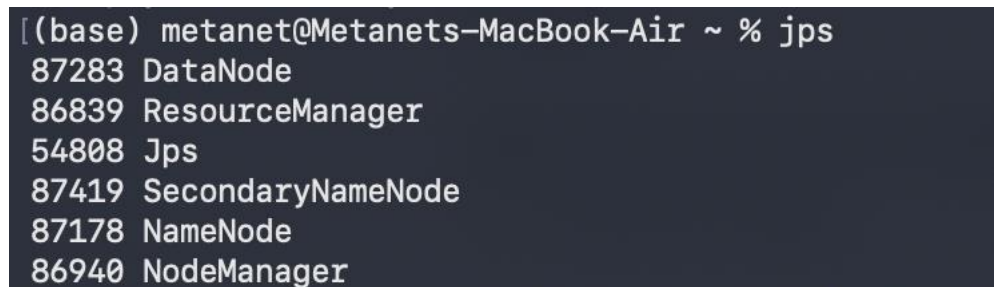
Additionally, I used external resources which included video tutorials to help me with the installation process. While general steps were the same, but some operations were different because of my system architecture.

This is the link to YouTube video: <https://www.youtube.com/watch?v=inDC9jgwpWY>

2. Part 2

Introduction:

In this part, I have used Hadoop and MapReduce to count the number of words in a text document (book.txt). The purpose was to understand how Hadoop and MapReduce work. At first, I wanted to ensure that Hadoop works correctly. To check this, I used the **jps** command. It showed all the main components of HDFS and YARN are running.



```
[(base) metanet@Metanets-MacBook-Air ~ % jps
87283 DataNode
86839 ResourceManager
54808 Jps
87419 SecondaryNameNode
87178 NameNode
86940 NodeManager
```

Data Preparation:

Firstly, I downloaded book.txt, then uploaded it into HDFS so Hadoop can do the tasks I want to do. Firstly, I created folder for input, for this I used this command:

```
"hdfs dfs -mkdir -p /user/$USER/input"
```

Then I added the file:

```
"hdfs dfs -put /Users/metanet/Downloads/book.txt /user/$USER/input/"
```

After that, I checked if file is uploaded, which proved that file is uploaded.

"hdfs dfs -ls /user/\$USER/input"

```
(base) metanet@Metanets-MacBook-Air ~ % hdfs dfs -ls /user/$USER/input
2026-04-05 12:21:02,860 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using
tin-java classes where applicable
Found 1 items
-rw-r--r--  1 metanet supergroup      106790 2026-04-05 12:20 /user/metanet/input/book.txt
```

Hadoop Configuration:

While setting up the Hadoop initially, I came across the problem of SSH configuration. It was related to that Remote Login option was turned off on the macOS, and therefore, Hadoop couldn't start normally. To solve this issue, I just activated this option via system settings. SSH connection then became possible, however, the system needed a password every time we accessed it, and this led to some problems. To solve this, I implemented passwordless SSH connection to system. As I did changes, everything was right, and Hadoop started working smoothly.

Running WordCount:

Before running program, I removed old output file (PS: I did once for testing):

"hdfs dfs -rm -r -f /user/\$USER/output/wordcount_out"

Then I ran WordCount :

"hadoop jar \$HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-3.3.6.jar wordcount /user/\$USER/input/book.txt /user/\$USER/output/wordcount_out"

Hadoop did its work and finished successfully

```
2026-04-05 12:22:52,679 INFO mapreduce.Job: map 100% reduce 100%
2026-04-05 12:22:52,681 INFO mapreduce.Job: Job job_local1436362361_0001 completed successfully
2026-04-05 12:22:52,689 INFO mapreduce.Job: Counters: 36
```

Checking Output:

After Hadoop finished work, I checked the output file:

"hdfs dfs -ls /user/\$USER/output/wordcount_out"

```
Bytes Written=47270
(base) metanet@Metanets-MacBook-Air ~ % hdfs dfs -ls /user/$USER/output/wordcount_out
[hdfs dfs -cat /user/$USER/output/wordcount_out/part-r-00000 | head
2026-04-05 12:23:00,125 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using
tin-java classes where applicable
Found 2 items
-rw-r--r--  1 metanet supergroup      0 2026-04-05 12:22 /user/metanet/output/wordcount_out/_SUCCESS
-rw-r--r--  1 metanet supergroup    47296 2026-04-05 12:22 /user/metanet/output/wordcount_out/part-r-00000
2026-04-05 12:23:00,713 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using
tin-java classes where applicable
#1", 1
#516, 1
#78306] 1
$5,000) 1
& 2
($1 1
(10,000 1
(862) 1
(International 1
(Kegan 1
```

Results:

This command showed the total numbers of words which is 17827:

```
" hdfs dfs -cat /user/$USER/output/wordcount_out/part-r-00000 | awk '{sum += $2} END {print sum}' "
```

This command showed the unique number of words which is 4486 :

```
" hdfs dfs -cat /user/$USER/output/wordcount_out/part-r-00000 | wc -l "
```

This command showed the top 10 most frequent words which are:

the 978 times

of 749 times

to 496 times

and 383 times

is 377 times

in 340 times

a 333 times

which 219 times

are 173 times

it 171 times

```
" hdfs dfs -cat /user/$USER/output/wordcount_out/part-r-00000 | sort -k2 -nr | head -10 "
```

```
(base) metanet@Metanets-MacBook-Air ~ % hdfs dfs -cat /user/$USER/output/wordcount_out/part-r-00000 | awk '{sum += $2} END {print sum}'
2026-04-05 12:23:46,030 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
17827
(base) metanet@Metanets-MacBook-Air ~ % hdfs dfs -cat /user/$USER/output/wordcount_out/part-r-00000 | wc -l
2026-04-05 12:23:58,664 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
4486
(base) metanet@Metanets-MacBook-Air ~ % hdfs dfs -cat /user/$USER/output/wordcount_out/part-r-00000 | sort -k2 -nr | head -10
the 978
of 749
to 496
and 383
is 377
in 340
a 333
which 219
are 173
it 171
(base) metanet@Metanets-MacBook-Air ~ %
```

Overall number of words: 17827

Number of unique words: 4486

10 most frequent words: the,of,to,and,is,in,a,which,are,it

I also observed some punctuation problems. For example, (Kegan ; \$5000), #1. It is probably, WordCount program does not clean text before counting.

3. Part 3

In this part, I refined the Wordcount MapReduce job that I did in Part 2 by eliminating stop words before doing the word count process. With the help of elimination of these common stopwords, the result would be much more meaningful and significant.

Firstly, I downloaded "stopword_list.txt." This file contained a list of stopwords including "the," "and," "is," "in," "to," etc.

For this part, I wrote another script mapper_stopwords.py. It included the following steps: load stopword list into the memory; iterate through the lines of the input text; make lowercase of all the words in each line; split the line by word. As to each word, the algorithm should verify whether the word belongs to the stopword list. If the word does not belong to the stopword list, it emits the word in pairs (word, 1). To refine the output, some pre-processing was done. First, it was necessary to lowercase all the words since "The" and "the" should be counted as one word. Also, all special symbols since "word," and "word" should be treated as one word. Only the alphabetical words were considered, and unnecessary parts such as the header and footer of the Project Gutenberg book were omitted.

```
(base) metanet@Metanets-MacBook-Air ~ % cat mapper_stopwords.py
#!/usr/bin/env python3
import sys
import re

# load stopwords
stopwords = set()
with open("stopword_list.txt", "r") as f:
    for line in f:
        stopwords.add(line.strip().lower())

inside = False
for line in sys.stdin:
    line_low = line.lower()

    # detect start of real book
    if "=== start of" in line_low:
        inside = True
        continue

    # stop at end
    if "=== end of" in line_low:
        break

    if not inside:
        continue

    # extract clean words only
    words = re.findall(r"[a-z]+(?:'[a-z]+)?", line_low)

    for word in words:
        if word not in stopwords:
            print(f"{word}\t1")
```

The reducer was similar to Part 2. It groups words with the same name and calculates the sum of their appearances.

```
(base) metanet@Metanets-MacBook-Air ~ % cat reducer.py
#!/usr/bin/env python3
import sys

current_word = None
current_count = 0

for line in sys.stdin:
    word, count = line.strip().split("\t")
    count = int(count)

    if word == current_word:
        current_count += count
    else:
        if current_word:
            print(f"{current_word}\t{current_count}")
        current_word = word
        current_count = count

if current_word:
    print(f"{current_word}\t{current_count}")
```


After creating the script files, I executed the Hadoop streaming job. As part of process, the `-files` parameter was used to ensure that the mapper and reducer scripts and the stop words file are available during execution.

```
(base) metanet@Metanets-MacBook-Air ~ % hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-*.jar \
-files mapper_stopwords.py, reducer.py, stopword_list.txt \
-mapper mapper_stopwords.py \
-reducer reducer.py \
-input /user/metanet/input/book.txt \
-output /output_part3
```

As job completed successfully, the output files were created on HDFS. First, the output data was displayed using:

```
(base) metanet@Metanets-MacBook-Air ~ % hdfs dfs -cat /output_part3/part-00000 | head -20
2026-04-05 17:33:42,545 WARN util.NativeCodeLoader: Unable to load native-hadoop library for y
tin-java classes where applicable
#1", 1
#516, 1
#78306] 1
$5,000) 1
& 2
($1 1
(10,000 1
(862) 1
(_e.g._, 1
(_tractatus 1
(a) 1
(an 1
(and 1
(any 1
(as 1
(b) 1
(c) 1
(compare 1
(does 1
(due 1
```

To identify the most frequent meaningful words, I sorted the final output in descending order by word count. The top 10 most frequently used non-stopword words are shown below

```
(base) metanet@Metanets-MacBook-Air ~ % hdfs dfs -cat /output_part3/part-00000 | sort -k2,2nr | head -10
2026-04-05 19:20:50,695 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform.
tin-java classes where applicable
poetry 98
attitudes 54
experience 53
words 47
may 45
one 42
us 40
mr 38
life 35
mind 33
(base) metanet@Metanets-MacBook-Air ~ % hdfs dfs -cat /output_part3/part-00000 | awk '1{sum += $2} END {print sum}'
```

As a result, the most common words found in the text include "attitudes", "poetry", and "experience". In comparison with the output from Part 2, common words such as "the", "is", and "in" are no longer present.

To calculate the total number of words after removing stopwords, I summed all the word frequencies from the final output file using a command. The result shows the total number of meaningful words remaining after filtering.

```
(base) metanet@Metanets-MacBook-Air ~ % hdfs dfs -cat /output_part3/part-00000 | awk '{sum += $2} END {print sum}'
2026-04-05 19:20:58,451 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using
a classes where applicable
6983
```

To calculate the number of unique words after removing stopwords, I counted the number of lines in the final output file. Since each line represents a distinct word, this gives the total number of unique non-stopword words in the text.

```
(base) metanet@Metanets-MacBook-Air ~ % hdfs dfs -cat /output_part3/part-00000 | wc -l
2026-04-05 19:21:05,232 WARN util.NativeCodeLoader: Unable to load native-hadoop library
tin-java classes where applicable
2556
```

To conclude, it was shown that the process of stop word removal can have a positive effect on text analysis results.

Summary

Total Number of Words after removing stopwords : 6983

Top 10 Most Frequently Used Words:

poetry	98
attitudes	54
experience	53
words	47
may	45
one	42
us	40
mr	38
life	35
mind	33

Unique words: 2556

Insights: Following the elimination of stopwords, the word counts help give insight into the real content of the text. Common words like “the”, “is”, and “in” are eliminated, making relevant words to be highlighted. It is clear from this example that the words that the most used include those that is related to major idea. This helps show how data cleaning helps improve data quality and discover new insights that were not easy to see before. In other words, data veracity will be achieved through data cleaning by eliminating irrelevant data. In summary, the improved results focus more on the main topics of the text than simple word counting methods

4. Link to Youtube video:

<https://www.youtube.com/watch?v=zAs-UMjXlxs>